Document number: P0539R1
Project: Programming Language C++
Audience: SG6 Numerics

Igor Klevanets <cerevra@yandex.ru>, <cerevra@yandex-team.ru>
Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2017-06-05

A Proposal to add wide_int Template Class

Significant changes to P0539R0 are marked with blue. Show/Hide deleted lines from P0275R0

Green lines are notes for the **editor** or for the **SG6** that must not be treated as part of the wording.

I. Introduction and Motivation

Current standard provides signed and unsigned int8_t, int16_t, int32_t, int64_t. It is usually enough for every day tasks, but sometimes appears a need in big numbers: for cryptography, IPv6, very big counters etc. Non-standard type __int128 which is provided by gcc and clang illuminates this need. But there is no cross-platform solution and no way to satisfy future needs in even more big numbers.

This is an attempt to solve the problem in a generic way on a library level and provide wording for P0104R0: Multi-Word Integer Operations and Types.

A proof of concept implementation available at: https://github.com/cerevra/int/tree/master/v3.

II. Changelog

Differences with P0539R0:

- Reworked the proposal for simpler integration with other Numerics proposals:
  - Added an interoperability section [numeric.interop]
  - `Arithmetic` and `Integral` concepts were moved to [numeric.requirements] as they seem widely useful
  - Binary non-member operations that accept `Arithmetic` or `Integral` parameter were changed to accept two `Arithmetic` or `Integral` parameters respectively and moved to [numeric.interop]
  - `int128_t` and other aliases now depend on P0102R0
- Renamed `wide_int` to `wide_integer` (this change is not highlighted with blue)
- `wide_integer` now uses machine words count as a template parameter, not bits count
- Removed not allowed type traits specializations
- Added `to_chars` and `from_chars`

III. Proposed wording

[Note to SG6: Following 4 paragraphs affect **all** the new numeric class proposals. ***- end note]***

18.3.4 Class template numeric_limits [numeric.limits]

[Note to editor: Add the following sentence after the sentence "Specializations shall be provided for each arithmetic type, both floating-point and integer, including bool." (first sentence in fourth paragraph in [numeric.limits]) ***- end note]***

Specializations shall be also provided for `wide_integer` type. *[Note:* If there is a built-in integral type `Integral` that has the same signedness and width as `wide_integer<MachineWords, S>`, then `numeric_limits<wide_integer<MachineWords, S>>` specialized in the same way as `numeric_limits<Integral>`- *end note]*

26.2 Definitions [numerics.defns]

[Note to editor: Add the following paragraph to the end of [numerics.defns] ***- end note]***

Define `CT` as `common_type_t<A, B>`, where `A` and `B` are the types of the two function arguments.

26.3 Numeric type requirements [numeric.requirements]

[Note to editor: Add the following paragraphs to the end of [numeric.requirements] ***- end note]***

Functions that accept template parameters starting with `Arithmetic` shall not participate in overload resolution unless `std::numeric_limits<Arithmetic>::is_specialized` is true.

Functions that accept template parameters starting with `Integral` shall not participate in overload resolution unless `std::numeric_limits<Integral>::is_integer` is true.

26.4 Numeric types interoperability [numeric.interop]

[Note to SG6: This is an early attempt to make it possible for different numeric types to inter-operate. It works for `wide_integer@built-in`, but this approach requires additional study and `common_type` fixes for new `Arithmetic` types. ***- end note]***

Following operators are defined for types `T` and `U` if `T` and `U` have a defined `common_type_t<T, U>` and satisfy the `Arithmetic` or `Integral` requirements from [numeric.requirements] *[Note:* Implementations are encouraged to provide optimized specializations of the following operators - *end note]*:

[Note to SG6: We may add to `Arithmetic` and `Integral` concepts additional requirement that `std::numeric_limits<`*`Integral-or-Arithmetic`*`>::is_interoprable` shall be true. In that case no user code will be broken even if user already added following operators. However this seems to be an overkill. ***- end note]***

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator*(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) * CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator/(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) / CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator+(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) + CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator-(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) - CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator%(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) % CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator&(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) & CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator|(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) | CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator^(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) ^ CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, size_t>
  constexpr operator<<(const Integral1& lhs, const Integral2& rhs);
```

Returns: `common_type_t<Integral1, size_t>(lhs) << static_cast<size_t>(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, size_t>
  constexpr operator>>(const Integral1& lhs, const Integral2& rhs);
```

Returns: `common_type_t<Integral1, size_t>(lhs) >> static_cast<size_t>(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator<(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) < CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator>(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) > CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator<=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) <= CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator>=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) >= CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator==(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) == CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator!=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `!(lhs == rhs)`.

26.??.1 Header <wide_integer> synopsis [numeric.wide_integer.syn]

```
namespace std {
  enum class signedness {
      Unsigned,
      Signed
  };

  // 26.??.2 class template wide_integer
  template<size_t MachineWords, signedness S> class wide_integer;

  // 26.??.?? type traits specializations
  template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  struct common_type<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>;

  template<size_t MachineWords, signedness S, typename Arithmetic>
  struct common_type<wide_integer<MachineWords, S>, Arithmetic>;

  template<typename Arithmetic, size_t MachineWords, signedness S>
  struct common_type<Arithmetic, wide_integer<MachineWords, S>>
    : common_type<wide_integer<MachineWords, S>, Arithmetic>
  ;

  // 26.??.?? unary operations
  template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator~(const wide_integer<MachineWords, S>& val) noexcept;
  template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator-(const wide_integer<MachineWords, S>& val) noexcept(S == signedness::Unsigned);
  template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator+(const wide_integer<MachineWords, S>& val) noexcept(S == signedness::Unsigned);

  // 26.??.?? binary operations
  template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
```

```
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator*(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator/(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator+(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept(S == signedness::Unsigned);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator-(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept(S == signedness::Unsigned);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator%(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator&(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator|(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr  operator^(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S>
  common_type_t<wide_integer<MachineWords, S>, size_t>
  constexpr  operator<<(const wide_integer<MachineWords, S>& lhs, size_t rhs);

template<size_t MachineWords, signedness S>
  common_type_t<wide_integer<MachineWords, S>, size_t>
  constexpr  operator>>(const wide_integer<MachineWords, S>& lhs, size_t rhs);

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator<(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator>(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator<=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator>=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator==(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator!=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;

// 26.??.?? numeric conversions
template<size_t MachineWords, signedness S> std::string to_string(const wide_integer<MachineWords, S>& val);
template<size_t MachineWords, signedness S> std::wstring to_wstring(const wide_integer<MachineWords, S>& val);

// 26.??.?? iostream specializations
template<class Char, class Traits, size_t MachineWords, signedness S>
basic_ostream<Char, Traits>& operator<<(basic_ostream<Char, Traits>& os, const wide_integer<MachineWords, S>& val);

template<class Char, class Traits, size_t MachineWords, signedness S>
basic_istream<Char, Traits>& operator>>(basic_istream<Char, Traits>& is, wide_integer<MachineWords, S>& val) noexcept;

// 26.??.?? hash support
template<class T> struct hash;
template<size_t MachineWords, signedness S> struct hash<wide_integer<MachineWords, S>>;
```

```
template <size_t MachineWords, signedness S>
to_chars_result to_chars(char* first,
                         char* last,
                         const wide_integer<MachineWords, S>& value,
                         int base = 10);

template <size_t MachineWords, signedness S>
from_chars_result from_chars(const char* first,
                             const char* last,
                             wide_integer<MachineWords, S>& value,
                             int base = 10);

template <size_t MachineWords>
using wide_int = wide_integer<MachineWords, signedness::Signed>;

template <size_t MachineWords>
using wide_uint = wide_integer<MachineWords, signedness::Unsigned>
```

[Note to SG6: Following part requires updated P0102R0 proposal with `exact_2int`, `fast_2int`, `least_2int`, `exact_2uint`, `fast_2uint`, `least_2uint` specializations for `wide_integer`. ***- end note]***

```
// optional aliases
using int128_t  = exact_2int<128>;      // wide_int<platform-specific> or platform-specific build in type
using uint128_t = exact_2uint<128>;     // wide_uint<platform-specific> or platform-specific build in type

using int256_t  = exact_2int<256>;      // wide_int<platform-specific> or platform-specific build in type
using uint256_t = exact_2uint<256>;     // wide_uint<platform-specific> or platform-specific build in type

using int512_t  = exact_2int<512>;      // wide_int<platform-specific> or platform-specific build in type
using uint512_t = exact_2uint<512>;     // wide_uint<platform-specific> or platform-specific build in type


// non optional aliases
using int_fast128_t  = fast_2int<128>;   // wide_int<platform-specific>
using uint_fast128_t = fast_2uint<128>;  // wide_uint<platform-specific>

using int_fast256_t  = fast_2int<256>;   // wide_int<platform-specific>
using uint_fast256_t = fast_2uint<256>;  // wide_uint<platform-specific>

using int_fast512_t  = fast_2int<512>;   // wide_int<platform-specific>
using uint_fast512_t = fast_2uint<512>;  // wide_uint<platform-specific>


using int_least128_t  = least_2int<128>;   // wide_int<platform-specific>
using uint_least128_t = least_2uint<128>;  // wide_uint<platform-specific>

using int_least256_t  = least_2int<256>;   // wide_int<platform-specific>
using uint_least256_t = least_2uint<256>;  // wide_uint<platform-specific>

using int_least512_t  = least_2int<512>;   // wide_int<platform-specific>
using uint_least512_t = least_2uint<512>;  // wide_uint<platform-specific>


// optional literals
inline namespace literals {
```

```
  inline namespace wide_int_literals {

  constexpr int128_t operator "" _int128(const char*);
  constexpr int256_t operator "" _int256(const char*);
  constexpr int512_t operator "" _int512(const char*);
  constexpr uint128_t operator "" _uint128(const char*);
  constexpr uint256_t operator "" _uint256(const char*);
  constexpr uint512_t operator "" _uint512(const char*);

  } // namespace wide_int_literals
  } // namespace literals
}
```

The header <wide_integer> defines class template `wide_integer` and a set of operators for representing and manipulating integers of specified width.

```
[Example:
    constexpr int128_t c = std::numeric_limits<int128_t>::min();
    static_assert(c == 0x80000000000000000000000000000000_uint128);

    int256_t a = 13;
    a += 0xFF;
    a *= 2.0;
    a -= 12_int128;
    assert(a > 0);
]
```

26.??.2 Template class wide_integer overview [numeric.wide_integer.overview]

```
namespace std {
  template<size_t MachineWords, signedness S>
  class wide_integer {
  public:
    // 26.??.2.?? construct:
    constexpr wide_integer() noexcept = default;
    template<typename Arithmetic> constexpr wide_integer(const Arithmetic& other) noexcept;
    template<size_t MachineWords2, signedness S2> constexpr wide_integer(const wide_integer<MachineWords2, S2>& other) noexcept;

    // 26.??.2.?? assignment:
    template<typename Arithmetic>
    constexpr wide_integer<MachineWords, S>& operator=(const Arithmetic& other) noexcept;
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator=(const wide_integer<MachineWords2, S2>& other) noexcept;

    // 26.??.2.?? compound assignment:
    template<typename Arithmetic>
    constexpr wide_integer<MachineWords, S>& operator*=(const Arithmetic&);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator*=(const wide_integer<MachineWords2, S2>&);

    template<typename Arithmetic>
    constexpr wide_integer<MachineWords, S>& operator/=(const Arithmetic&);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator/=(const wide_integer<MachineWords2, S2>&);

    template<typename Arithmetic>
    constexpr wide_integer<MachineWords, S>& operator+=(const Arithmetic&) noexcept(S == signedness::Unsigned);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator+=(const wide_integer<MachineWords2, S2>&) noexcept(S == signedness::Unsigned);

    template<typename Arithmetic>
    constexpr wide_integer<MachineWords, S>& operator-=(const Arithmetic&) noexcept(S == signedness::Unsigned);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator-=(const wide_integer<MachineWords2, S2>&) noexcept(S == signedness::Unsigned);

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator%=(const Integral&);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator%=(const wide_integer<MachineWords2, S2>&);

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator&=(const Integral&) noexcept;
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator&=(const wide_integer<MachineWords2, S2>&) noexcept;

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator|=(const Integral&) noexcept;
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator|=(const wide_integer<MachineWords2, S2>&) noexcept;

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator^=(const Integral&) noexcept;
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator^=(const wide_integer<MachineWords2, S2>&) noexcept;

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator<<=(const Integral&);
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator<<=(const wide_integer<MachineWords2, S2>&);

    template<typename Integral>
    constexpr wide_integer<MachineWords, S>& operator>>=(const Integral&) noexcept;
    template<size_t MachineWords2, signedness S2>
    constexpr wide_integer<MachineWords, S>& operator>>=(const wide_integer<MachineWords2, S2>&) noexcept;

    constexpr wide_integer<MachineWords, S>& operator++() noexcept(S == signedness::Unsigned);
    constexpr wide_integer<MachineWords, S> operator++(int) noexcept(S == signedness::Unsigned);
    constexpr wide_integer<MachineWords, S>& operator--() noexcept(S == signedness::Unsigned);
    constexpr wide_integer<MachineWords, S> operator--(int) noexcept(S == signedness::Unsigned);

    // 26.??.2.?? observers:
    template <typename Arithmetic> constexpr operator Arithmetic() const noexcept;
    constexpr explicit operator bool() const noexcept;

  private:
    uint32_t data[MachineWords / sizeof(uint32_t) + 1]; // exposition only
  };
}
```

The class template `wide_integer<size_t MachineWords, signedness S>` is a POD class that behaves as an integer type of a compile time specified width. Template parameter `MachineWords` specifies exact *machine words* count to store the integer value. `sizeof(wide_integer<MachineWords, signedness::Unsigned>)` and `sizeof(wide_integer<MachineWords, signedness::Signed>)` are required to be equal to `MachineWords * sizeof(`*machine-word*`)`. Template parameter `S` specifies signedness of the stored integer value.

[Note to SG6: Definition for *machine words* must be provided somewhere and referenced from the above paragraph. ***- end note]***

26.??.2.?? wide_integer constructors [numeric.wide_integer.cons]

```
constexpr wide_integer() noexcept = default;
```

Effects: A Constructs an object with undefined value.

```
template<typename Arithmetic> constexpr wide_integer(const Arithmetic& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

```
template<size_t MachineWords2, signedness S2> constexpr wide_integer(const wide_integer<MachineWords2, S2>& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

26.??.2.?? wide_integer assignments [numeric.wide_integer.assign]

```
template<typename Arithmetic>
constexpr wide_integer<MachineWords, S>& operator=(const Arithmetic& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

```
template<size_t MachineWords2, signedness S2>
constexpr wide_integer<MachineWords, S>& operator=(const wide_integer<MachineWords2, S2>& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

26.??.2.?? wide_integer compound assignments [numeric.wide_integer.cassign]

```
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator*=(const wide_integer<MachineWords2, S2>&);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator/=(const wide_integer<MachineWords2, S2>&);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator+=(const wide_integer<MachineWords2, S2>&) noexcept(S == signedness::Unsigned);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator-=(const wide_integer<MachineWords2, S2>&) noexcept(S == signedness::Unsigned);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator%=(const wide_integer<MachineWords2, S2>&);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator&=(const wide_integer<MachineWords2, S2>&) noexcept;
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator|=(const wide_integer<MachineWords2, S2>&) noexcept;
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator^=(const wide_integer<MachineWords2, S2>&) noexcept;
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator<<=(const wide_integer<MachineWords2, S2>&);
template<size_t MachineWords2, signedness S2> constexpr wide_integer<MachineWords, S>& operator>>=(const wide_integer<MachineWords2, S2>&) noexcept;
constexpr wide_integer<MachineWords, S>& operator++() noexcept(S == signedness::Unsigned);
constexpr wide_integer<MachineWords, S> operator++(int) noexcept(S == signedness::Unsigned);
constexpr wide_integer<MachineWords, S>& operator--() noexcept(S == signedness::Unsigned);
constexpr wide_integer<MachineWords, S> operator--(int) noexcept(S == signedness::Unsigned);
```

Effects: Behavior of the above operators is similar to operators for built-in integral types.

```
template<typename Arithmetic> constexpr wide_integer<MachineWords, S>& operator*=(const Arithmetic&);
template<typename Arithmetic> constexpr wide_integer<MachineWords, S>& operator/=(const Arithmetic&);
template<typename Arithmetic> constexpr wide_integer<MachineWords, S>& operator+=(const Arithmetic&) noexcept(S == signedness::Unsigned);
template<typename Arithmetic> constexpr wide_integer<MachineWords, S>& operator-=(const Arithmetic&) noexcept(S == signedness::Unsigned);
```

Effects: As if an object `wi` of type `wide_integer<MachineWords, S>` was created from input value and the corresponding operator was called for `*this` and the `wi`.

```
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator%=(const Integral&);
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator&=(const Integral&) noexcept;
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator|=(const Integral&) noexcept;
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator^=(const Integral&) noexcept;
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator<<=(const Integral&);
template<typename Integral> constexpr wide_integer<MachineWords, S>& operator>>=(const Integral&) noexcept;
```

Effects: As if an object `wi` of type `wide_integer<MachineWords, S>` was created from input value and the corresponding operator was called for `*this` and the `wi`.

26.??.2.?? wide_integer observers [numeric.wide_integer.observers]

```
template <typename Arithmetic> constexpr operator Arithmetic() const noexcept;
```

Returns: If `Arithmetic` type is an integral type then it is constructed from `*this` using the integral conversion rules [conv.integral]. Otherwise `Arithmetic` is constructed from `*this` using the floating-integral conversion rules [conv.fpint].

```
constexpr explicit operator bool() const noexcept;
```

Returns: `true` if `*this` is not equal to 0.

26.??.?? Specializations of common_type [numeric.wide_integer.traits.specializations]

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
struct common_type<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>> {
  using type = wide_integer<max(MachineWords, MachineWords2), see below>;
};
```

The signed template parameter indicated by this specialization is following:

- `(S == signedness::Signed && S2 == signedness::Signed ? signedness::Signed : signedness::Unsigned)` if `MachineWords == MachineWords2`
- `S` if `MachineWords > MachineWords2`
- `S2` otherwise

[Note: `common_type` follows the *usual arithmetic conversions* design. - *end note]*

[Note to SG6: `common_type` attempts to follow the *usual arithmetic conversions* design here for interoperability between different numeric types. Following two specializations must be moved to a more generic place and enriched with usual arithmetic conversion rules for **all the other** numeric classes that specialize `std::numeric_limits`- ***end note]***

```
template<size_t MachineWords, signedness S, typename Arithmetic>
struct common_type<wide_integer<MachineWords, S>, Arithmetic> {
  using type = see below;
};
```

```
template<typename Arithmetic, size_t MachineWords, signedness S>
struct common_type<Arithmetic, wide_integer<MachineWords, S>>
  : common_type<wide_integer<MachineWords, S>, Arithmetic>;
```

The member typedef `type` is following:

- `Arithmetic` if `numeric_limits<Arithmetic>::is_integer` is `false`
- `wide_integer<MachineWords, S>` if `sizeof(wide_integer<MachineWords, S>) > sizeof(Arithmetic)`
- `Arithmetic` if `sizeof(wide_integer<MachineWords, S>) < sizeof(Arithmetic)`
- `Arithmetic` if `sizeof(wide_integer<MachineWords, S>) == sizeof(Arithmetic) && S == signedness::Signed`
- `Arithmetic` if `sizeof(wide_integer<MachineWords, S>) == sizeof(Arithmetic) && numeric_limits<wide_integer<MachineWords, S>>::is_signed == numeric_limits<Arithmetic>::is_signed`

- `wide_integer<MachineWords, S>` otherwise

26.??.?? Unary operators [numeric.wide_integer.unary_ops]

```
template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator~(const wide_integer<MachineWords, S>& val) noexcept;
```

Returns: value with inverted significant bits of `val`.

```
template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator-(const wide_integer<MachineWords, S>& val) noexcept(S == signedness::Unsigned);
```

Returns: `val *= -1` if `S` is true, otherwise the result is unspecified.

```
template<size_t MachineWords, signedness S> constexpr wide_integer<MachineWords, S> operator+(const wide_integer<MachineWords, S>& val) noexcept(S == signedness::Unsigned);
```

Returns: `val`.

26.??.?? Binary operators [numeric.wide_integer.binary_ops]

In the function descriptions that follow, `CT` represents `common_type_t<A, B>`, where `A` and `B` are the types of the two arguments to the function.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator*(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);
```

Returns: `CT(lhs) *= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator/(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);
```

Returns: `CT(lhs) /= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator+(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept(S == signedness::Unsigned);
```

Returns: `CT(lhs) += rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator-(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept(S == signedness::Unsigned);
```

Returns: `CT(lhs) -= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator%(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs);
```

Returns: `CT(lhs) %= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator&(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `CT(lhs) &= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr operator|(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `CT(lhs) |= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  common_type_t<wide_integer<MachineWords, S>, wide_integer<MachineWords2, S2>>
  constexpr  operator^(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `CT(lhs) ^= rhs`.

```
template<size_t MachineWords, signedness S>
  common_type_t<wide_integer<MachineWords, S>, size_t>
  constexpr operator<<(const wide_integer<MachineWords, S>& lhs, size_t rhs);
```

Returns: `CT(lhs) <<= rhs`.

```
template<size_t MachineWords, signedness S>
  common_type_t<wide_integer<MachineWords, S>, size_t>
  constexpr operator>>(const wide_integer<MachineWords, S>& lhs, size_t rhs) noexcept;
```

Returns: `CT(lhs) >>= rhs`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator<(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `true` if value of `CT(lhs)` is less than the value of `CT(rhs)`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator>(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `true` if value of `CT(lhs)` is greater than the value of `CT(rhs)`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator<=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `true` if value of `CT(lhs)` is equal or less than the value of `CT(rhs)`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator>=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `true` if value of `CT(lhs)` is equal or greater than the value of `CT(rhs)`.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
  constexpr bool operator==(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `true` if significant bits of `CT(lhs)` and `CT(rhs)` are the same.

```
template<size_t MachineWords, signedness S, size_t MachineWords2, signedness S2>
```

```
constexpr bool operator!=(const wide_integer<MachineWords, S>& lhs, const wide_integer<MachineWords2, S2>& rhs) noexcept;
```

Returns: `!(CT(lhs) == CT(rhs))`.

26.??.?? Numeric conversions [numeric.wide_integer.conversions]

```
template<size_t MachineWords, signedness S> std::string to_string(const wide_integer<MachineWords, S>& val);
template<size_t MachineWords, signedness S> std::wstring to_wstring(const wide_integer<MachineWords, S>& val);
```

Returns: Each function returns an object holding the character representation of the value of its argument. All the significant bits of the argument are outputed as a signed decimal in the style `[-]dddd`.

```
template <size_t MachineWords, signedness S>
to_chars_result to_chars(char* first,
                         char* last,
                         const wide_integer<MachineWords, S>& value,
                         int base = 10);
```

Behavior of `wide_integer` overload is subject to the usual rules of primitive numeric output conversion functions [utility.to.chars].

```
template <size_t MachineWords, signedness S>
from_chars_result from_chars(const char* first,
                             const char* last,
                             wide_integer<MachineWords, S>& value,
                             int base = 10);
```

Behavior of `wide_integer` overload is subject to the usual rules of primitive numeric input conversion functions [utility.from.chars].

26.??.?? iostream specializations [numeric.wide_integer.io]

```
template<class Char, class Traits, size_t MachineWords, signedness S>
basic_ostream<Char, Traits>& operator<<(basic_ostream<Char, Traits>& os, const wide_integer<MachineWords, S>& val);
```

Effects: As if by: `os << to_string(val)`.

Returns: `os`.

```
template<class Char, class Traits, size_t MachineWords, signedness S>
basic_istream<Char, Traits>& operator>>(basic_istream<Char, Traits>& is, wide_integer<MachineWords, S>& val);
```

Effects: Extracts a `wide_integer` that is represented as a decimal number in the `is`. If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios::failure` ([iostate.flags])).

Returns: `is`.

26.??.?? Hash support [numeric.wide_integer.hash]

```
template<size_t MachineWords, signedness S> struct hash<wide_integer<MachineWords, S>>;
```

The specialization is enabled (20.14.14). If there is a built-in integral type `Integral` that has the same signedness and width as `wide_integer<MachineWords, S>`, and `wi` is an object of type `wide_integer<MachineWords, S>`, then `hash<wide_integer<MachineWords, S>>()(wi) == hash<Integral>()(Integral(wi))`.

IV. Feature-testing macro

For the purposes of SG10we recommend the feature-testing macro name `__cpp_lib_wide_integer`.